Synapse AI End-to-End Performance Report

Date: April 23, 2026 System: Synapse AI / TrustMed AI Source data: `results/e2e_performance_report_20260423T214459Z.json` Primary test patient: `10002428`

Executive Summary

Synapse AI was tested end to end across the FastAPI backend, Next.js frontend, patient data access, graph retrieval, vector retrieval, upload handling, panel detection, patient attachments, streaming chat, SOAP note generation, browser route loading, and quality gates.

The system passed all sampled endpoint, streaming, browser, component, and validation checks. MedGemma and Vertex AI paths were intentionally excluded, as requested. The test instead used `nvidia/nemotron-3-nano-30b-a3b:free` for text-backed samples and `google/gemini-3-flash-preview` for the non-MedGemma vision sample.

Overall result: Passed.

Scope

Tested surfaces:

- API health and cache management
- Session creation, retrieval, rename, list, and cleanup
- Patient chart retrieval from SQLite
- Patient attachment listing and upload
- PDF report text extraction
- Clinician image upload and patient-linking flow
- Panel detection endpoint
- Knowledge graph endpoint
- Medical term explanation endpoint
- Patient summary generation
- Full non-streaming RAG chat
- Streaming patient chat
- SOAP note generation
- Non-MedGemma vision chat path
- Mixed concurrent API load
- Lightweight scope-guard stream load
- Desktop and mobile browser route loads
- Frontend production build
- Patient attachment pytest suite
- JSON syntax validation

Excluded surfaces:

- Vertex AI MedGemma text endpoint
- Vertex AI MedGemma vision endpoint

MedGemma exclusion controls:

- Backend was started with `VERTEX_PROJECT_ID`, `VERTEX_ENDPOINT_ID`, `VERTEX_DEDICATED_DOMAIN`, and `VERTEX_SERVICE_ACCOUNT_JSON` blanked.
- No request payload used `vertex/medgemma-27b-it`.
- Non-MedGemma vision testing used `google/gemini-3-flash-preview`.

Data Store Readiness

Data store	Result	Evidence
SQLite MIMIC demo database	Passed	`vitalsign`: 1,038; `diagnosis`: 545; `medrecon`: 2,764; `triage`: 222; `edstays`: 222; `pyxis`: 1,082

Data store	Result	Evidence
Chroma vector store	Passed	`symptoms`: 3,932; `medicines`: 3,881; `diseases`: 1,702; `medical_images`: 4,199

Performance Results

Test class	Passed	Success rate	Median latency	p95 latency	Max latency
API endpoint samples	24 / 24	100.00%	3.33 ms	29,437.47 ms	33,332.14 ms
Streaming samples	2 / 2	100.00%	30,477.90 ms	39,487.37 ms	40,488.42 ms
Browser route loads	8 / 8	100.00%	713.25 ms	900.38 ms	913.56 ms
Direct component checks	3 / 3	100.00%	734.14 ms	1,131.04 ms	1,175.14 ms

Load Tests

Load test	Requests	Workers	Throughput	Median latency	p95 latency	Result
Mixed lightweight API load	60	10	61.907 req/s	7.35 ms	930.43 ms	Passed
Patient scope-guard stream load	12	6	526.170 req/s	10.39 ms	11.62 ms	Passed

Notable Endpoint Timings

Flow	Latency	Result
API health check, warm sample	1.17 ms	Passed
Patient record fetch	3.80 ms	Passed
Knowledge graph query with patient context	574.44 ms	Passed
Image upload	2.85 ms	Passed
Panel detection	18.40 ms	Passed
PDF attachment upload and local extraction	12.09 ms	Passed
Patient summary generation	12,084.34 ms	Passed
Full non-streaming RAG chat	27,194.89 ms	Passed
SOAP note generation	8,356.79 ms	Passed
Non-MedGemma vision chat	33,332.14 ms	Passed

Browser Route Results

The production Next.js app was tested in headless Chromium at desktop and mobile viewport sizes.

Route	Desktop wall time	Mobile wall time	Console warnings/errors	Result
`/`	913.56 ms	772.12 ms	0	Passed
`/clinician`	875.89 ms	853.90 ms	0	Passed
`/patient`	589.74 ms	583.85 ms	0	Passed
`/patient/about`	654.38 ms	600.66 ms	0	Passed

Quality Gates

Gate	Result	Runtime
Frontend production build, `npm run build`	Passed	9,798.39 ms

Gate	Result	Runtime
Patient attachment pytest suite	Passed, 14 tests	8,470.26 ms
JSON report syntax validation	Passed	44.14 ms

Interpretation

The core local application surfaces are responsive. Simple backend endpoints generally complete in single-digit milliseconds once warm. Browser route loads stayed below one second across the tested desktop and mobile viewports.

The largest latencies are concentrated in model-backed and multimodal flows. This is expected because full chat, streaming synthesis, SOAP generation, and vision analysis depend on external model calls, graph context, safety review, and, for vision, local visual-RAG model loading. These flows passed, but they should be treated as seconds-scale interactions rather than instant UI operations.

The patient scope-guard load test is especially strong when the guard triggers correctly, with p95 latency of 11.62 ms across 12 concurrent stream requests.

Observed Issues

Two issues were discovered and filed in `bd` for follow-up.

Issue	Severity	Evidence	Impact
`TrustMed-AI-i3u`: backend Chroma path is startup-directory-sensitive	High	API chat samples launched from `api/` logged missing `medicines`, `diseases`, and `symptoms` collections. Direct component testing from the repo root found the populated Chroma collections and returned 30,198 characters in 1,175.14 ms.	Full chat still completed through graph, patient, and model paths, but vector retrieval was effectively absent under the documented backend startup path.
`TrustMed-AI-23s`: one patient off-topic travel prompt missed the lightweight scope guard	Medium	A patient-mode vacation-planning stream entered patient context, graph context, drug checking, and model synthesis, taking 40,488.42 ms. Simpler unrelated prompts short-circuited with p95 11.62 ms.	Some unrelated patient questions can pay full model and retrieval latency instead of returning the intended lightweight scope response.

Limitations

- MedGemma and Vertex AI were intentionally excluded.
- Model-backed timing depends on OpenRouter availability and rate limits at the time of testing.
- Browser measurements were local headless Chromium runs, not distributed real-user monitoring.
- Load tests were controlled local samples, not cloud-scale stress tests.

Conclusion

The system passed the end-to-end performance and availability test suite outside the explicitly excluded MedGemma paths. The frontend, backend, patient data access, graph endpoint, attachment workflows, PDF extraction, streaming responses, SOAP generation, browser routes, and validation gates all completed successfully.

The system is presentable with two caveats: fix the Chroma path issue before claiming full vector retrieval in the documented backend startup mode, and tighten the patient off-topic guard to avoid unnecessary heavy model calls for unrelated prompts.

Generated from results/clean_e2e_performance_report_20260423.md and raw benchmark JSON in results/e2e_performance_report_20260423T214459Z.json.